

chap 01 : 데이터 다루기

Data Science Basic

소프트웨어 융합공학과

남춘성

경고문구

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. 약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다.

학생의 불법 유통으로 저작권자가 저작권 침해 소송을 제기한 경우, 불법유통시킨 학생의 법적책임만 문제될 뿐, 교수님의 법적책임은 인정되지 않음을 안내드립니다.

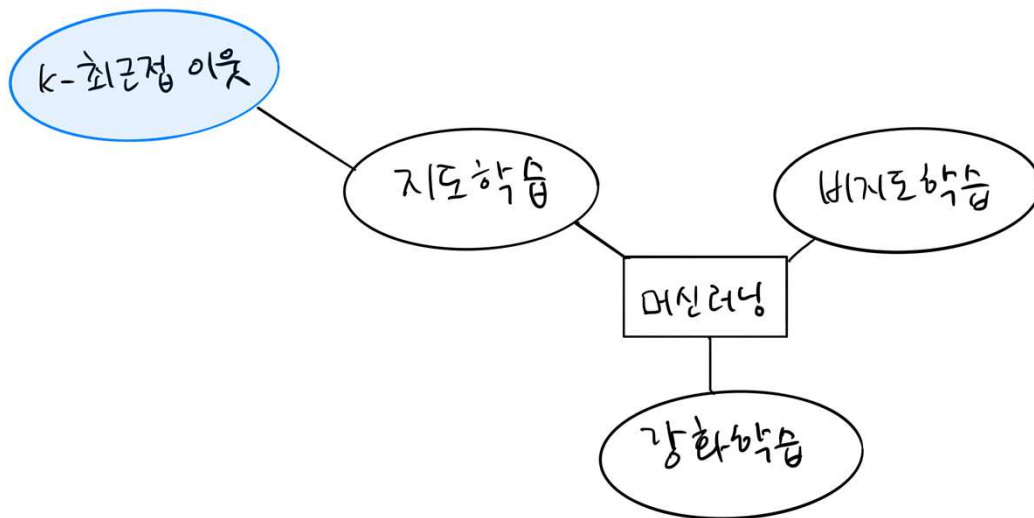
2025. 9. 인하대학교·한국복제전송저작권협회

1-1 머신러닝을 위한 환경 설정

- 자신의 환경에 맞게 환경이 설정
 - Anaconda
 - VS Code
 - Google colab
 - 등등
- GPU 사용가능 환경은 선택
 - ML(Machine Learning)에서는 무의미
 - CPU로만 사용가능

2-1. 지도 학습과 비지도 학습

- 지도 학습(supervised learning)
 - 정답(타겟)이 있으니 알고리즘이 정답을 맞추는 것을 학습
 - 훈련 데이터 : 입력(input)-특성(feature) + 타겟(target)
- 비지도 학습(unsupervised learning)
 - 타겟없이 입력 데이터 만으로 정답을 학습



훈련 데이터

입력		타겟
길이 특성	무게 특성	
25.4	242.0	1
26.3	290.0	1
⋮	⋮	⋮
15.0	19.9	∅

49개 샘플

2-1. 훈련 세트와 테스트 세트

- 훈련세트(train set) : 훈련에 사용되는 데이터
- 테스트세트(test set) : 평가에 사용되는 데이터

```
fish_length = [25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7, 31.0, 31.0,  
               31.5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0, 34.0, 34.5, 35.0,  
               35.0, 35.0, 35.0, 36.0, 36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0, 9.8,  
               10.5, 10.6, 11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0]
```

```
\fish_weight = [242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0, 500.0, 475.0, 500.0,  
                500.0, 340.0, 600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0, 685.0, 620.0, 680.0,  
                700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0, 6.7,  
                7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.4, 12.2, 19.7, 19.9]
```

```
fish_data = [[l,w] for l, w in zip(fish_length, fish_weight)]  
Fisht_target = [1]*35+[0]*14
```

2-1. 훈련 세트와 테스트 세트

- KNN(K-Nearest Neighbor)

- 가장 가까운 이웃을 찾기 위해, distance measure를 찾음
 - Euclidean distance/Manhattan distance/Minkowski distance

```
from sklearn.neighbors import KNeighborsClassifier  
kn = KNeighborClassifier()
```

#sample 확인

```
print(fish_data[4])  
print(fish_data[0:5])  
print(fish_data[:5])  
print(fish_data[44:])
```

27개의 특성

49개 샘플

훈련 세트 35개

테스트 세트 14개

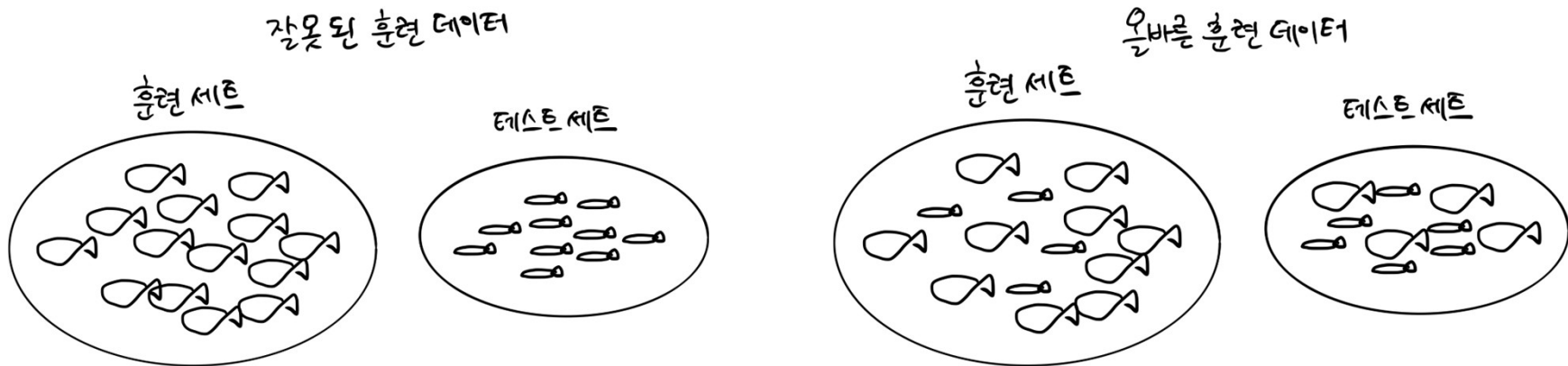
```
train_input = fish_data[:35]  
train_target = fish_target[:35]
```

```
test_input = fish_data[35:]  
test_target = fish_target[35:]
```

2-1. 훈련 세트와 테스트 세트

- 모델 훈련 (fit), 테스트 (score)

```
kn = kn.fit(train_input, train_target) # kn_model = kn.fit(train_input, train_target)
kn.score(test_input, test_target)
# 결과 0.0 : 샘플링 편향(sampling bias)
```



2-1. 훈련 세트와 테스트 세트

- 넘파이 이용하기

1차원 배열

→ 축 1

3	10	2	5	7	8	9
---	----	---	---	---	---	---

2차원 배열

→ 축 2

축 1 ↓

3	10	2	...			

3차원 배열

축 3 ↗

축 2 →

축 1 ↓

3	10	2	...			

```
import numpy as np
```

```
input_arr = np.array(fish_data)  
target_arr = np.array(fish_target)
```

```
print(input_arr)
```

배열형태 확인

```
print(input_arr.shape)  
#(49, 2)
```

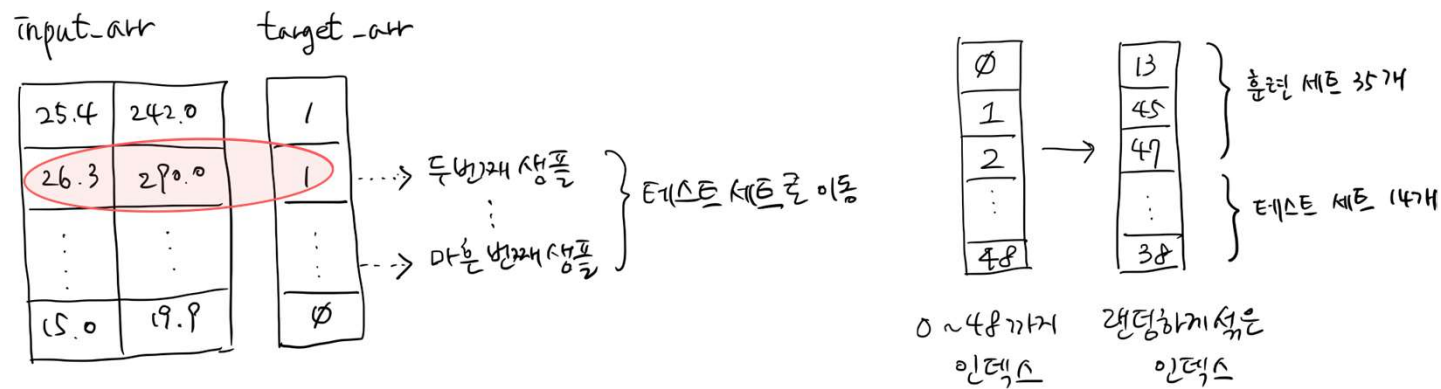
2개의 열 (특성)

49개
행(샘플)

{
 [25.4 , 242.0],
 [26.3 , 290.0],
 ⋮
 [15.0 , 19.9]
}

2-1. 훈련 세트와 테스트 세트

- 넘파이 이용하여 무작위 샘플 선택



```
np.random.seed(42)
```

```
index = np.arange(49)
```

```
np.random.shuffle(index)
```

```
[13 45 47 44 17 27 26 25 31 19 12  4 34  8  3  6 40 41 46 15  9 16 24 33
 30  0 43 32  5 29 11 36  1 21  2 37 35 23 39 10 22 18 48 20  7 42 14 28
 38]
```

```
train_input = input_arr[index[:35]]
train_target = target_arr[index[:35]]
```

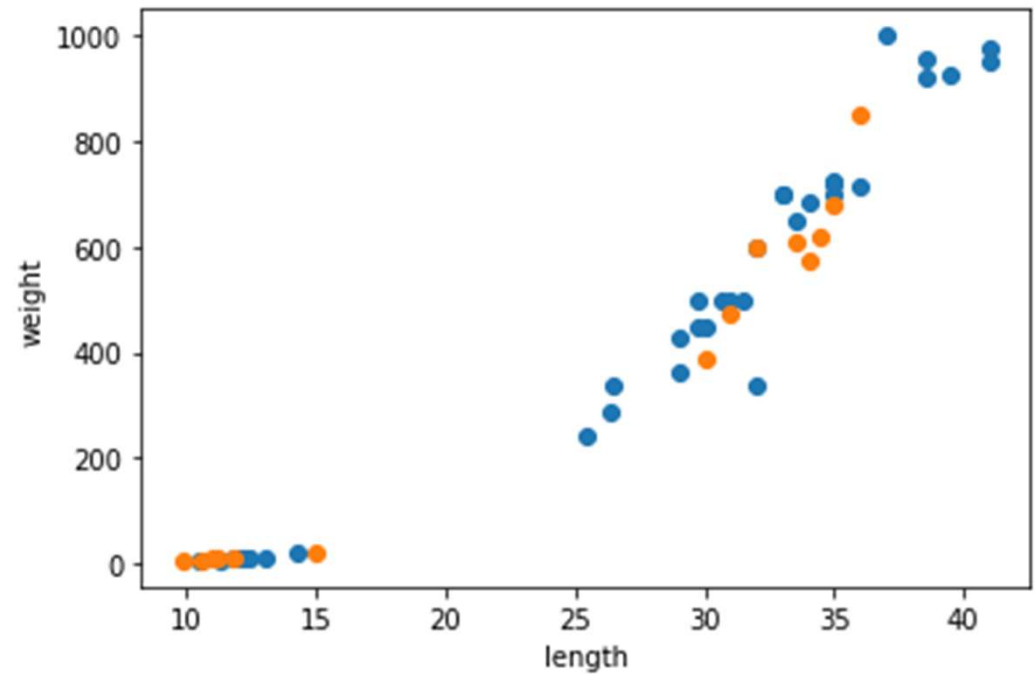
```
test_input = input_arr[index[35:]]
test_target = target_arr[index[35:]]
```

2-1. 훈련 세트와 테스트 세트

- 데이터 확인하기

```
import matplotlib.pyplot as plt

plt.scatter(train_input[:, 0], train_input[:, 1])
plt.scatter(test_input[:, 0], test_input[:, 1])
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```



2-1. 두 번째 머신러닝 프로그램

- 잘 섞은 데이터 기반 학습하기

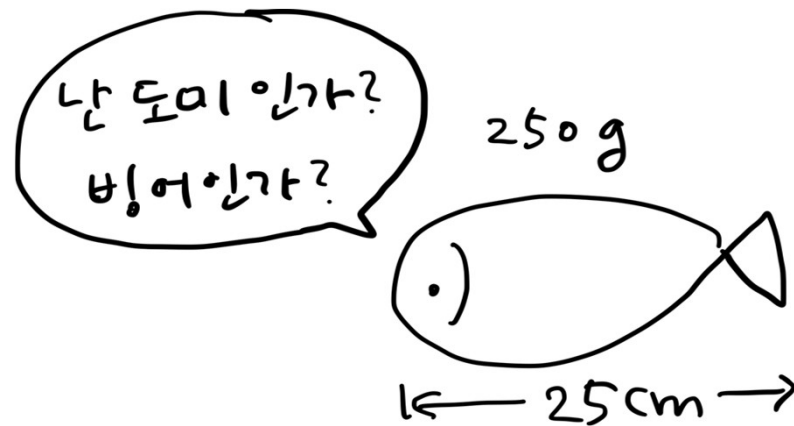
```
kn = kn.fit(train_input, train_target) # kn_model = kn.fit(train_input, train_target)
kn.score(test_input, test_target)
# 결과 1.0

kn.predict(test_input)
# array([0, 0, 1, 0, 1, 1,1, 0, 1,1, 0, 1,1, 0])

test_target
# array([0, 0, 1, 0, 1, 1,1, 0, 1,1, 0, 1,1, 0])
```

2-2. 데이터 전처리

- 도미인가? 빙어인가?
 - 학습한 모델은 빙어, 하지만 실제로는 도미
 - 학습한 모델이 길이와 무게를 가지고 있지만 생각해 보면 길이와 무게는 단위가 다름



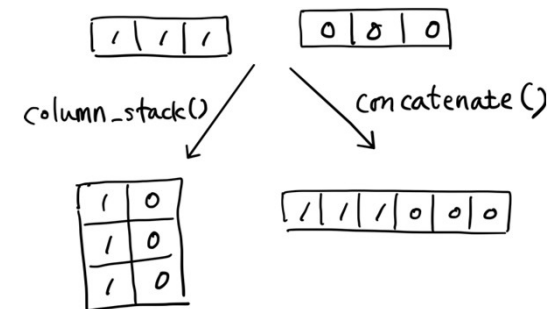
2-2. 데이터 전처리

- 넘파이로 데이터 준비하기
- `np.column_stack`을 이용하여 1:1 매칭하여 `fish_data` 데이터 변경
 - `Fish_lengt`와 `fish_weigh`를 `array([[25.4, 242], ...])`형태로 변경
- `Np.concatenate`을 이용하여 `target` 생성 후 합침

```
fish_data = np.column_stack((fish_length, fish_weight))
```

$$\begin{bmatrix} 25.4 & 242. \\ 26.3 & 290. \\ 26.5 & 340. \\ 29. & 363. \\ 29. & 430. \end{bmatrix}$$

```
fish_target = np.concatenate((np.ones(35), np.zeros(14)))
```

[illegible]

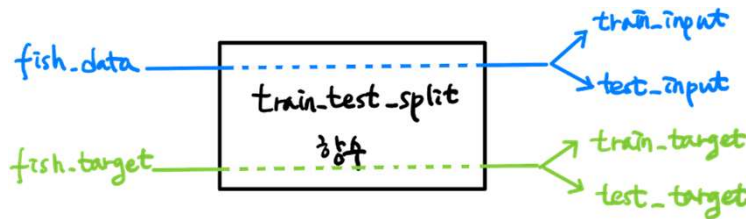
2-2. 데이터 전처리

- 사이킷런으로 훈련 세트와 테스트 세트 나누기

```
from sklearn.model_selection import train_test_split
```

```
train_input, test_input, train_target, test_target = train_test_split(  
    fish_data, fish_target, stratify=fish_target, random_state=42)
```

stratify 속성으로 클래스 비율에 맞게 데이터를 나누게 됨



#shape 속성 알아내기

```
print(train_input.shape, test_input.shape)
```

```
# (36, 2) (13, 2)
```

```
print(train_target.shape, test_target.shape)
```

```
# (36,) (13, )
```

2-2. 데이터 전처리

- 수상한 도미 한마리

```
from sklearn.neighbors import KNeighborsClassifier
```

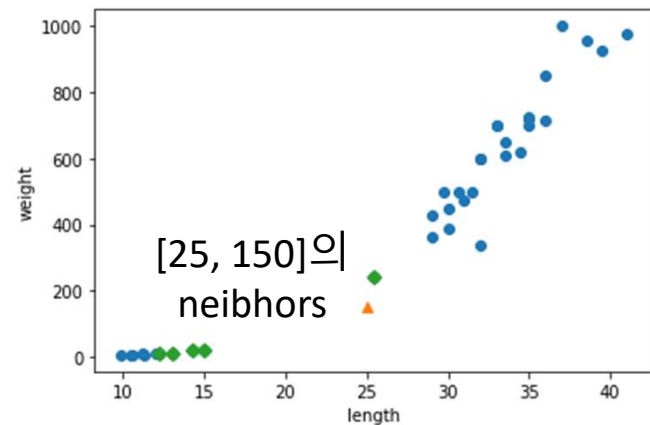
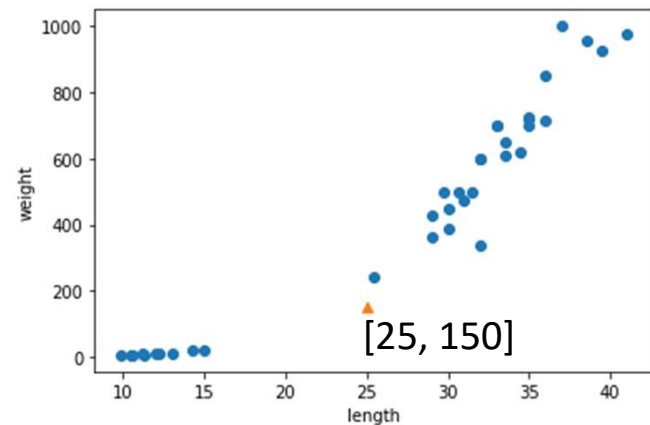
```
kn = KNeighborsClassifier()  
kn.fit(train_input, train_target)  
kn.score(test_input, test_target)  
1.0
```

```
print(kn.predict([[25, 150]]))  
[0.]
```

1이 되어야 함

```
distances, indexes = kn.kneighbors([[25, 150]])
```

```
plt.scatter(train_input[:,0], train_input[:,1])  
plt.scatter(25, 150, marker='^')  
plt.scatter(train_input[indexes,0],  
            train_input[indexes,1], marker='D')  
plt.xlabel('length')  
plt.ylabel('weight')  
plt.show()
```



2-2. 데이터 전처리

- 수상한 도미 한마리

- `print(train_input[indexes])` 를 통해 값을 확인할 수 있음

- 25.4 242. , 15. 19.9 , 14.3 19.7, 13. 19.7, 12.2 12.2

- `train_target[indexes]`

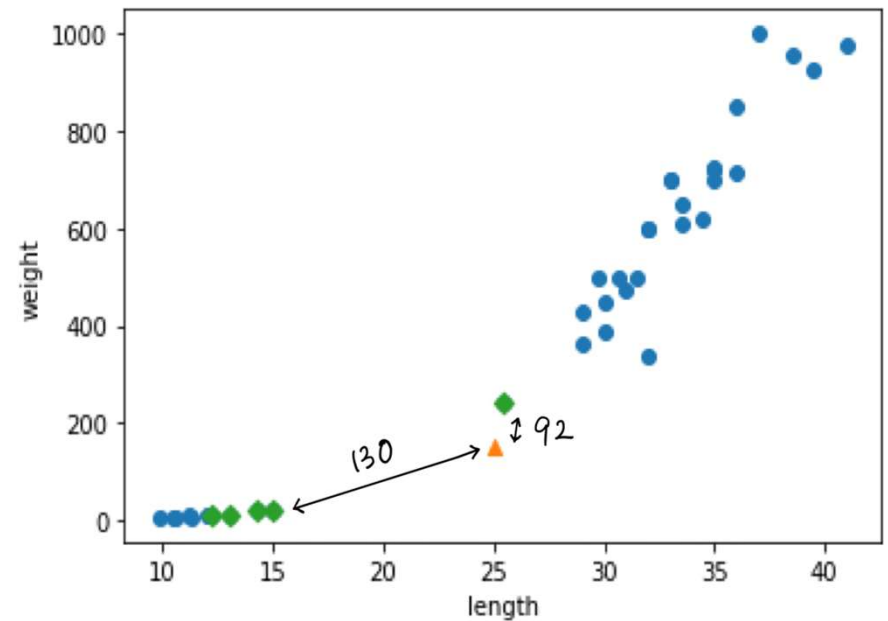
- 1, 0, 0, 0, 0

- distances

- 92.0000, 130.??????, 130.??????,.....

- 거리가 이상함

- 거리의 비율이 맞지 않음
 - 즉, X축과 Y축의 크기가 다르기 때문
 - 그래프로 확인하기 힘들

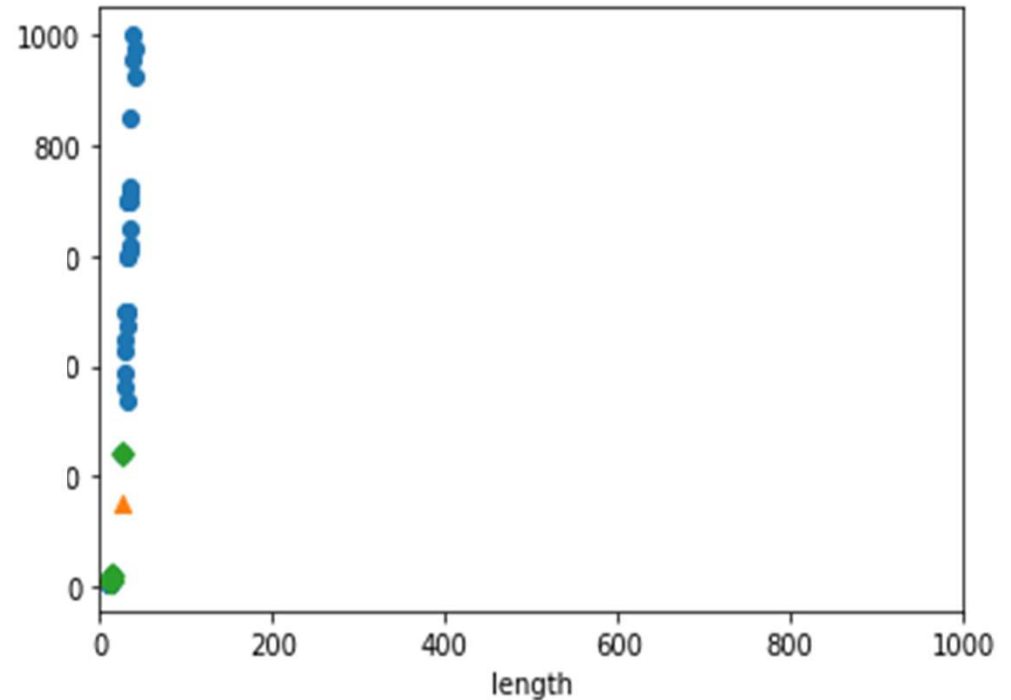


2-2. 데이터 전처리

- 수상한 도미 한마리
 - x축과 y축 범위 지정
 - Xlim(), ylim() .

```
plt.scatter(train_input[:,0], train_input[:,1])
plt.scatter(25, 150, marker='^')
plt.scatter(train_input[indexes,0],
            train_input[indexes,1], marker='D')
plt.xlim((0, 1000))
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```

- 스케일(scale)이 다름
 - 데이터전처리(data preprocessing) 해야함.



x축은 가장 가까운 이웃을 찾는데 큰 영향이 없음을 나타냄

2-2. 데이터 전처리

- 표준점수(standard score) : z 점수
 - 각 특성값이 0에서 표준편차의 몇 배만큼 떨어져 있는지를 나타냄
 - 실제 특성값의 크기와 상관없이 동일한 조건으로 비교 가능
 - 분산 : 데이터에서 평균을 뺀 값을 모두 제곱한 다음 평균을 내어 구함
 - 표준편차 : 분산의 제곱근으로 데이터가 분산된 정도
 - 표준점수 : 각 데이터가 원점에서 몇 표준편차만큼 떨어져 있는지 나타내는 값

2개 특성
→ axis=1

axis=0 ↓

29.7	500
12.2	12.2
.	.
.	.
.	.
41	995

367개 샘플

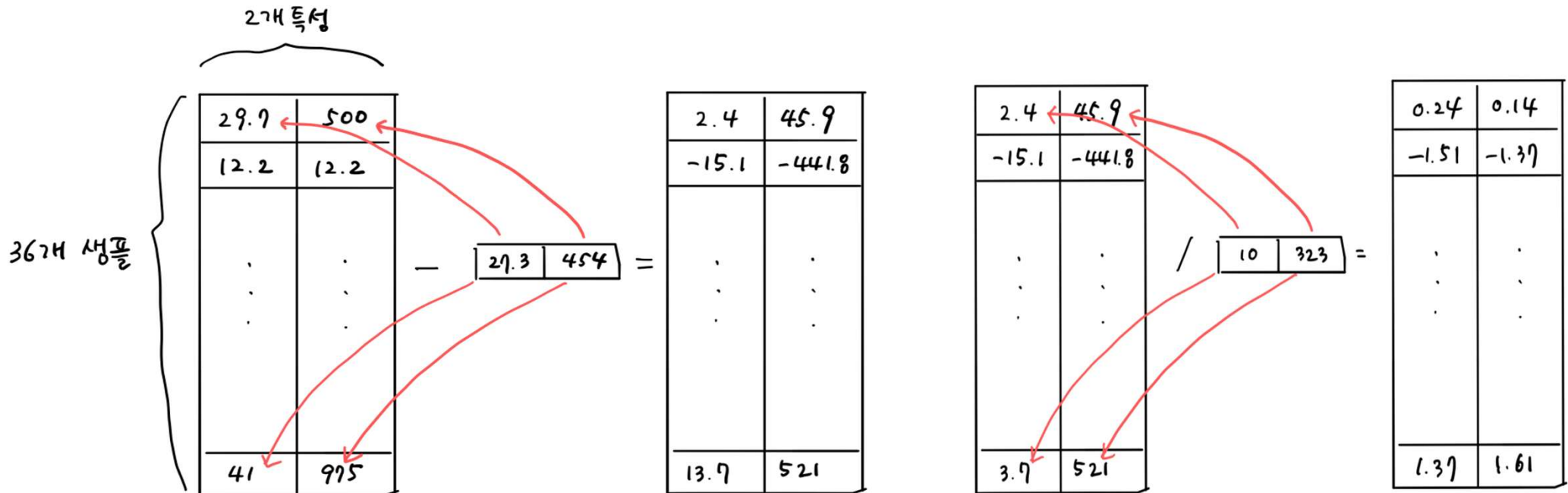
```
mean = np.mean(train_input, axis=0)
std = np.std(train_input, axis=0)

print(mean, std)
[ 27.29722222 454.09722222] [  9.98244253 323.29893931]

train_scaled = (train_input - mean) / std
```

2-2. 데이터 전처리

- Mean(평균)빼기 와 std(표준편차)나누기



2-2. 데이터 전처리

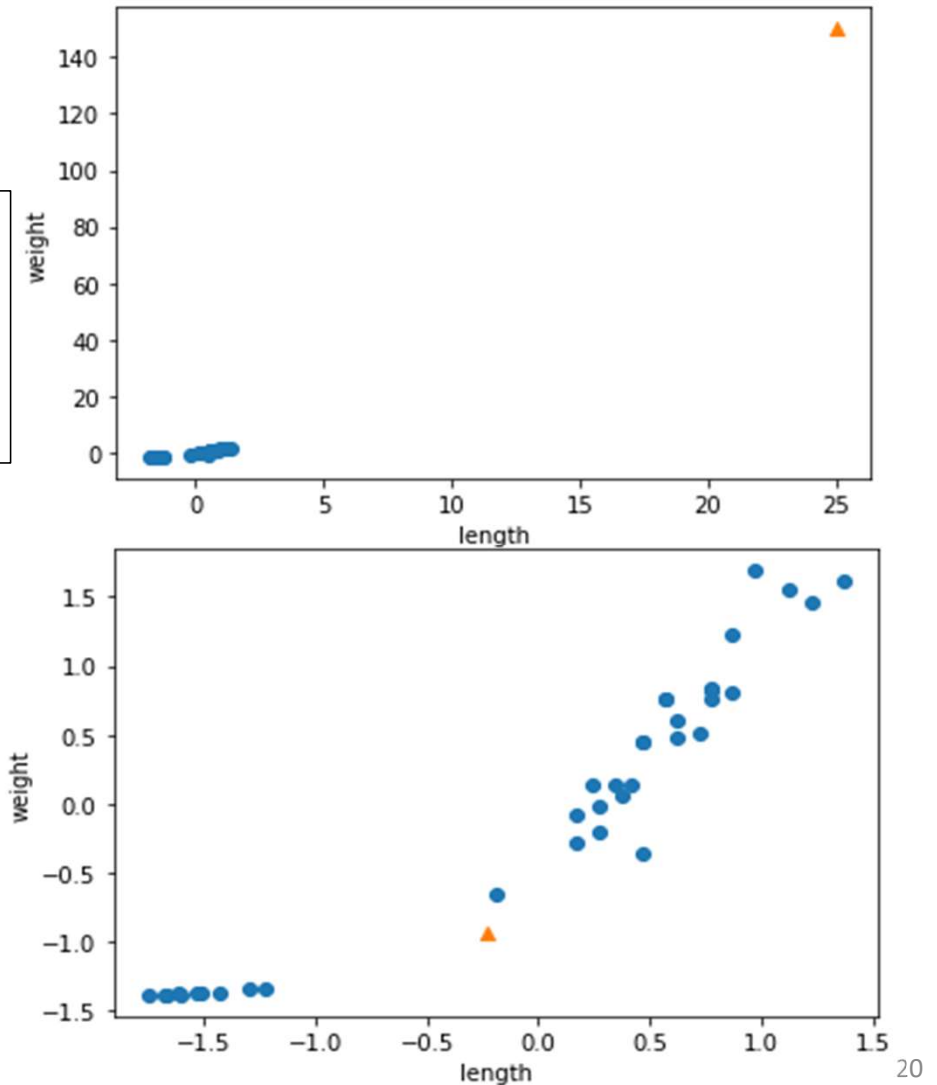
- 전처리 데이터로 모델 훈련하기

```
plt.scatter(train_scaled[:,0], train_scaled[:,1])  
plt.scatter(25, 150, marker='^')  
plt.xlabel('length')  
plt.ylabel('weight')  
plt.show()
```

```
new = ([25, 150] - mean) / std
```

```
plt.scatter(train_scaled[:,0], train_scaled[:,1])  
plt.scatter(new[0], new[1], marker='^')  
plt.xlabel('length')  
plt.ylabel('weight')  
plt.show()
```

범위 -1.5~1.5 사이



2-2. 데이터 전처리

- 전처리후 데이터 훈련

```
kn.fit(train_scaled, train_target)

test_scaled = (test_input - mean) / std
kn.score(test_scaled, test_target)
1.0

print(kn.predict([new]))
[1.]

distances, indexes = kn.kneighbors([new])

plt.scatter(train_scaled[:,0], train_scaled[:,1])
plt.scatter(new[0], new[1], marker='^')
plt.scatter(train_scaled[indexes,0],
train_scaled[indexes,1], marker='D')
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```

